



- **Intro to Testing**



Introduction

Testing is the process of finding errors

We can test code, implementations, documentation, and even other testing.

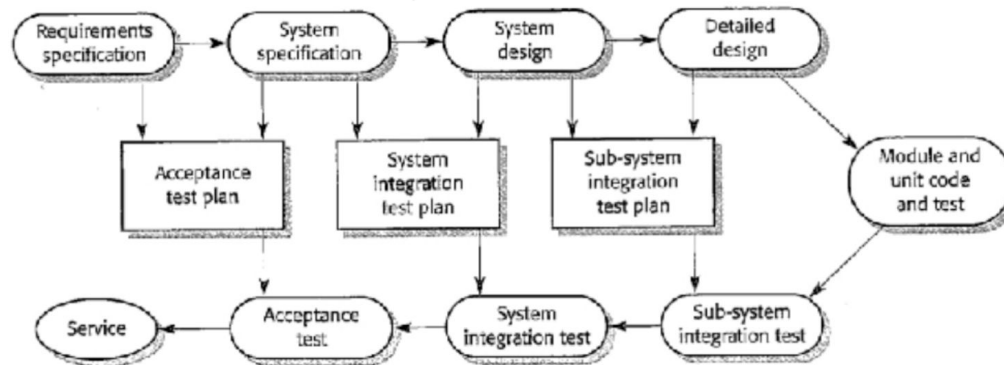
Test Data - Inputs that are designed to test the system.

Test Cases - Ways in which we operate the system with the given data.

Oracle - The set of "good" outcomes.

Reality Check

Why Should we test



Source: https://www.cs.uic.edu/~jbell/CourseNotes/OO_SoftwareEngineering/Testing.html



Crowdstrike Event 2024

On July 19, 2024, at exactly 4:09 AM UTC, a software update began rolling out automatically to millions of computers worldwide. By 5:27 AM — just 78 minutes later — approximately 8.5 million Windows machines were in a crash loop, showing the Blue Screen of Death. Airlines grounded flights. Hospitals reverted to paper. Banks went offline. 911 emergency services in some regions were disrupted. The fix? IT staff had to physically walk to each machine, boot into Safe Mode, and manually delete a single file

https://en.wikipedia.org/wiki/2024_CrowdStrike-related_IT_outages



What would you have tested differently ?

Discussion

Verification vs Validation

VERIFICATION

"Are we building
the product right?"

- Conforms to spec & design
- Internal quality checks
- Reviews, inspections, walkthroughs
- Unit & integration tests



CrowdStrike

CrowdStrike: Did the update pass the validator?

X No — the Content Validator had its own bug

VALIDATION

"Are we building
the right product?"

- Meets the user's actual need
- External reality check
- User acceptance testing
- End-to-end system tests



CrowdStrike

CrowdStrike: Should the update go to 100% of sensors at once?

X No — no staged rollout, no kill switch



So why aren't you doing it?

- Not enough time!
- Not a project requirement!
- Not something a developer should do!
- Not something I'm interested in!
- It's not worth it!



Types of Testing

- **Black Box Testing** - is designed solely based on the specifications of the unit, without any knowledge of its internal workings.
- **White box testing** takes advantage of knowledge of the internal structure of the unit being tested.
- **Functional Testing**: Testing this black box testing type is related to the functional requirements of a system
- **Non-Functional Testing** - This is a type of software testing that checks non-functional aspects such as performance usability reliability of a software application a great example of a non-functional test.
- **Regression Testing**: Whenever developers change or modify their software even a small change can have unexpected consequences regression testing involves testing existing software applications to ensure that a change or addition hasn't broken any existing functionality.

Black Box Testing Technique

Equivalence Class Testing: A black box test technique in which test scales are designed to exercise one representative member of each partition. For ex. Negative or positive number partition.

```
class MyGregorianCalendar {  
    ...  
    public static int getNumDaysInMonth(int month, int year) {...}  
    ...  
}
```

Figure 11-10 Interface for a method computing the number of days in a given month (in Java). The `getNumDaysInMonth()` method takes two parameters, a month and a year, both specified as integers.

Table 11-2 Equivalence classes and selected valid inputs for testing the `getNumDaysInMonth()` method.

Equivalence class	Value for month input	Value for year input
Months with 31 days, non-leap years	7 (July)	1901
Months with 31 days, leap years	7 (July)	1904
Months with 30 days, non-leap years	6 (June)	1901
Month with 30 days, leap year	6 (June)	1904
Month with 28 or 29 days, non-leap year	2 (February)	1901
Month with 28 or 29 days, leap year	2 (February)	1904

Boundary Testing

Values on the "edges" or boundaries of equivalence ranges are most likely to cause faults. For example, leap years have special rules for years divisible by 100 or 400. (Empty strings are often taken as a special or boundary case.)

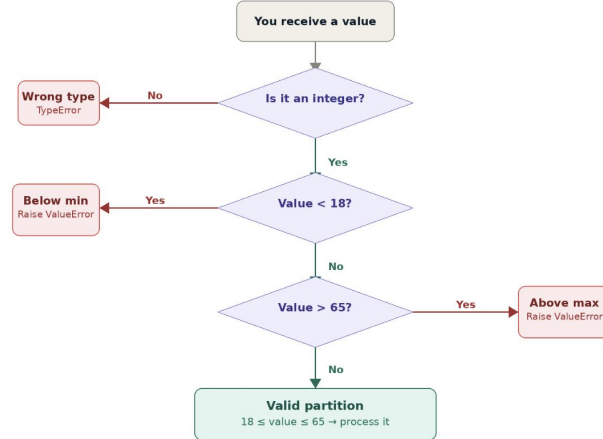
Table 11-3 Additional boundary cases selected for the `getNumDaysInMonth()` method.

Equivalence class	Value for month input	Value for year input
Leap years divisible by 400	2 (February)	2000
Non-leap years divisible by 100	2 (February)	1900
Nonpositive invalid months	0	1291
Positive invalid months	13	1315

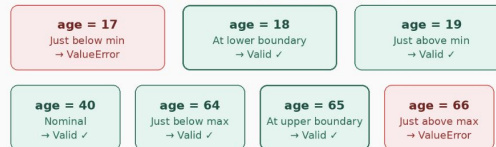
Boundary Value Testing Ex.

Boundary value analysis — what test case is this?

Function accepts age (integer). Valid range: 18-65.



The 7 test cases you must always write



Valid (accepted)

Invalid (rejected)



White Box Testing Strategies

Structural coverage can be used to quantify the thoroughness of testing

Complex coverage types may be more effective at detecting defects than simpler ones, but they are often associated with a tradeoff

- More costly to collect
- Achieving full coverage might not be practical

Structural Coverage Strategies (White Box Testing)

- Definition use Coverage
- Logic Coverage
- Statement Coverage
- Branch Coverage



Definition-use Coverage

A *definition-use* (or *def-use*) is defined with respect to some variable

- **Def**: location where v is defined
- **Use**: location where v is used

Can be used to define different testing criteria such as:

- Use every *def*
- Execute every *use*

Example

Compute the max of two numbers

```
s1. x = readInput();  
s2. y = readInput();  
s3. max = x;  
s4. if(y > max)  
s5.     max = y;  
s6. output(max);
```

Test case (x=5, y=1)

Statements	Branches	Def-uses
s1	s4→s6	(x, s1, s3)
s2		(y, s2, s4)
s3		(max, s3, s4)
s4		(max, s3, s6)
s6		



Logic Coverage Criteria

Predicate coverage

- The test set should make each predicate evaluate to true and false

Clause coverage

- The test set should make each clause evaluate to true and false

Active clause coverage

- Similar to clause coverage with the extra requirement that the clause should determine the predicate



Logic Coverage Example

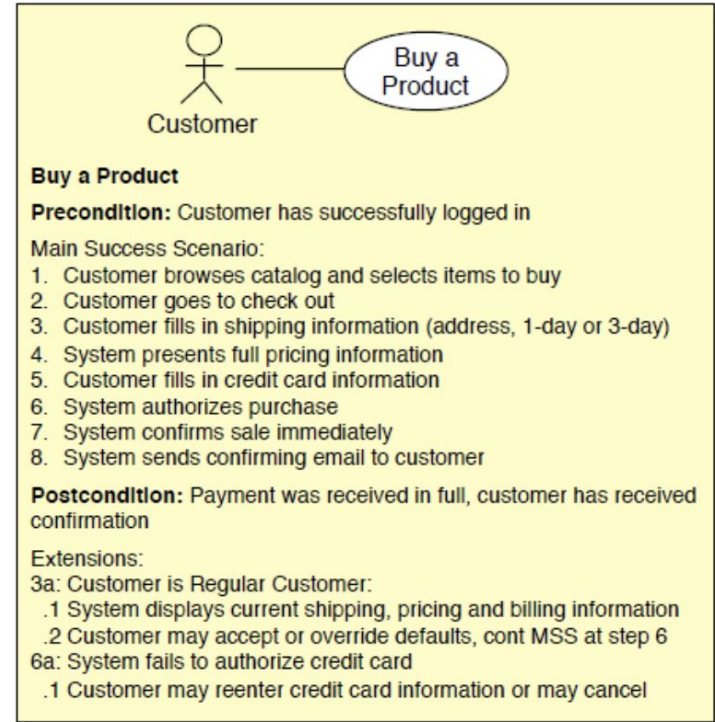
Expression: $(a > b) \ \&\& \ (c == 1 \ || \ d == 1)$

Tests: $(a=4, b=3, c=1, d=2)$, $(a=3, b=4, c=2, d=1)$

Which logic criteria are satisfied?

Generating test from Use Cases

- Test the basic flow
- Test the alternate flows
- Test the postconditions
- Break the preconditions - What happens if this is not met?
- Identify options for each input choice
 - Select combinations of options for each test case





Smoke Testing

- Smoke testing is an approach usually carried out during the initial stages of Software development lifecycle
- Main function to make sure core functionalities of the program are working fine without any issues
- Executed before any detailed functional test

Sanity Testing

- Sanity testing is performed to check whether the software product is working correctly
- Performed after receiving a stable build
- Quick evaluation of the quality of the software release to determine if it's eligible for further rounds of testing or not





Regression Testing

- Regression testing is the process of verifying that the software still performs correctly after changes were introduced
 - Bug fixes doesn't affect the previous build
- Generally, new test cases are not created and previous test cases are executed
- Once Sanity testing of the changed functionality is completed, all impacted features of the application requires complete testing
 - New features being introduced
 - Defects being fixed
 - Performances fixes
 - Change in requirement for existing software



Smoke Testing	Sanity Testing	Regression Testing
Performed on initial builds	Performed on stable builds	Performed on stable builds
Tests the stability of a new build	Tests the stability of a new functionality or code changes in the existing build	Tests the functionality of all affected areas after new functionality /code changes in the existing build
Covers end-to-end basic functionalities	Covers certain modules, in which code changes have been made	Covers detailed testing targeting all the affected areas after new functionalities are added
Executed by testers & sometimes also by developers	Executed by testers	Executed by testers, mostly via automation
A part of basic testing	A part of regression testing	Regression Testing is a superset of Smoke and Sanity Testing

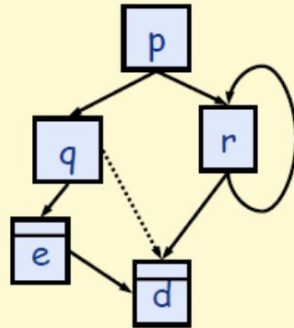
Integration Testing

- Units are tested together
- Two main approaches

Bottom up

for this dependency graph, test order is:

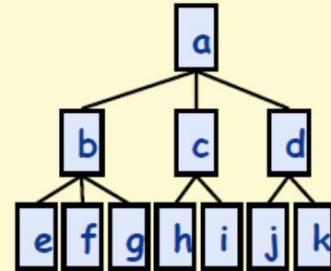
- 1) d
- 2) e and r
- 3) q
- 4) p



Top down

for this structure chart the order is:

- 1) test a with stubs for b, c, and d
- 2) test a+b+c+d with stubs for e...k
- 3) test whole system





End to End Testing

Simulate how real users use the application.

Current approach: Click the same sequence of buttons until it works.

New approach: A demo with Cypress!



Example - Test a pencil

A non software example to demonstrate creative test generation

Know the Customer & Product

- Ask thoughtful question to understand who and how the product will be used
- Customer - Know who will be using the product
 - School - art students ? how dark/ thick ? Age of student ?
 - Prison? Can't be too sharp :)
 - International ? What language, locales
- Product: Know the requirement / how the product will be used
 - Eraser?
 - Length ? if it will be sharpened or unsharpened
 - Color ?
 - How long to write before sharpening ?
- Now we know the SUT (system under test) and can begin generating test cases





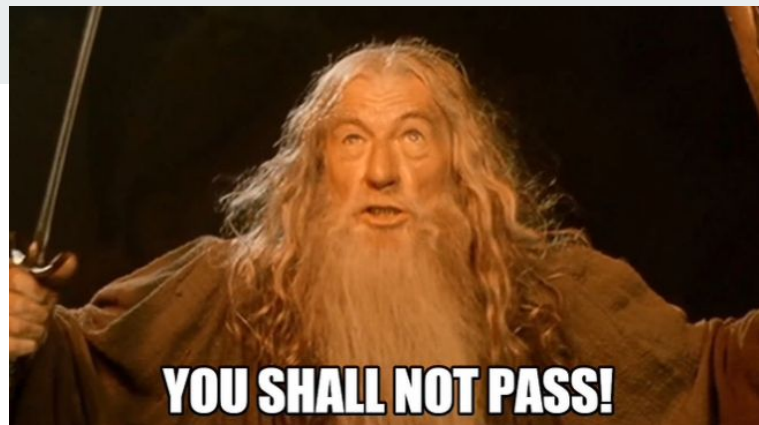
Creative Test Generation

- **Functional** - write on standard white lined paper, erase
- **Error** - write on skin, under water , metal , walls
- **Boundary** - write after 1st sharpen, shortest length , 1st erase , last erase
- **Performance** - how long to write before sharpening ? temperature , bend break point, tip pressure breakpoint (what's average human pressure), how much to erase before it's worn down, erase 1 month after writing
- **Security** - lead toxicity ?
- **Accessibility** - hand/finger sizes, feel different than other writing tools (blind sensory)
- **Internationalization** - color offensive to other cultures? How will ship ? packaging ? Instruction ? do all customer know how to use the pencil

Automated Testing

Continuous Integration Pipelines like Github Actions

- Run your test suite for each push to a PR, show immediate feedback.
- Demo!





Limitation of Testing

Cannot find bugs in code not executed, cannot assure absence of bugs

- Nondeterminism, flaky tests
- Certain kinds of bugs occur only under very unlikely conditions
- Hard to observe/assert specifications • Memory leaks, information flow, ...
- Potentially expensive, long run times
- Potentially high manual effort
- Verification, not validation

Are all your
unit tests
passing?

Yep

Do you have
100 %
code coverage?

Yep

Are there still going
to be unexplainable
bugs and reusability
issues?

Yep



**Low coverage means
insufficient testing.**



References

1. Bernd Bruegge and Allen H. Dutoit, "Object-Oriented Software Engineering", Third Edition, Chapter 11
2. Mauro Pezze and Michal Young, "Software Testing and Analysis - Process, Principles, and Techniques".
3. Ian Sommerville, "Software Engineering 8".



Thank You